

BCA-II SEMESTER

MATHEMATICAL FOUNDATIONS OF COMPUTER SCIENCE (BCA-108)

Unit II:

SYLLABUS :

Algorithm: Algorithms, merits and demerits, Exponentiation, How to compute fast exponentiation. Linear Search, Binary Search, "Big Oh" notation, Worst case, Advantage of logarithmic algorithms over linear algorithms, complexity.

=====

ALGORITHM

What is Algorithm | Introduction to Algorithms

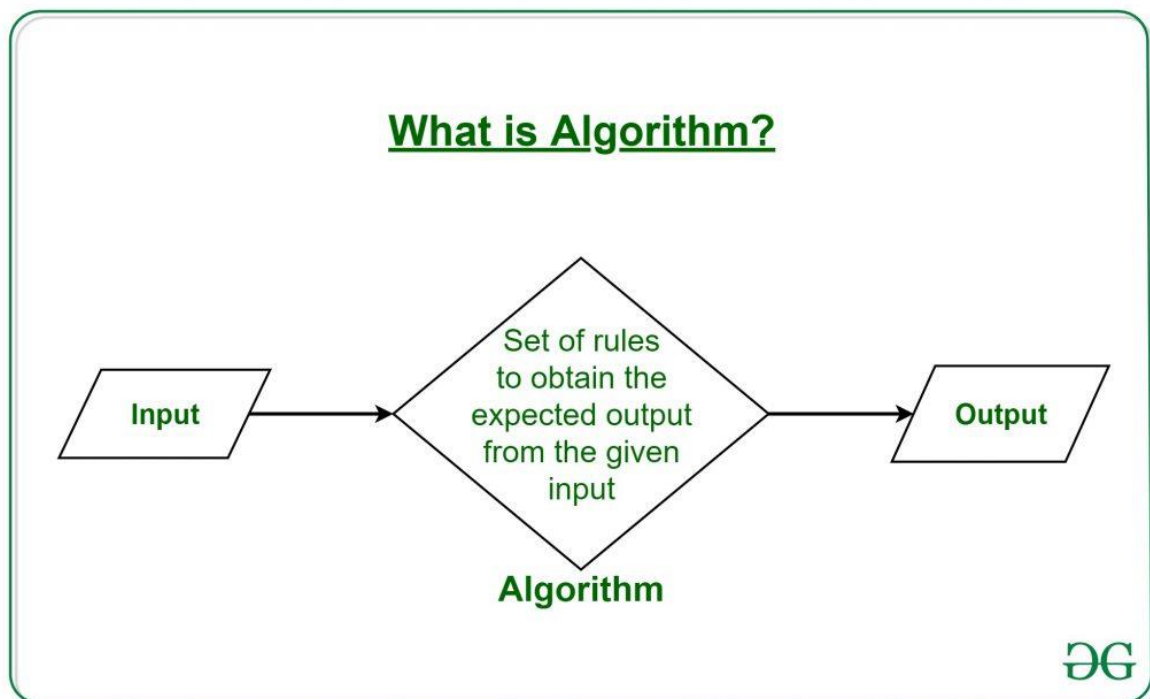
Definition of Algorithm

The word Algorithm means "A set of finite rules or instructions to be followed in calculations or other problem-solving operations"

Or

"A procedure for solving a mathematical problem in a finite number of steps that frequently involves recursive operations".

Therefore Algorithm refers to a sequence of finite steps to solve a particular problem.



Use of the Algorithms:

Algorithms play a crucial role in various fields and have many applications. Some of the key areas where algorithms are used include:

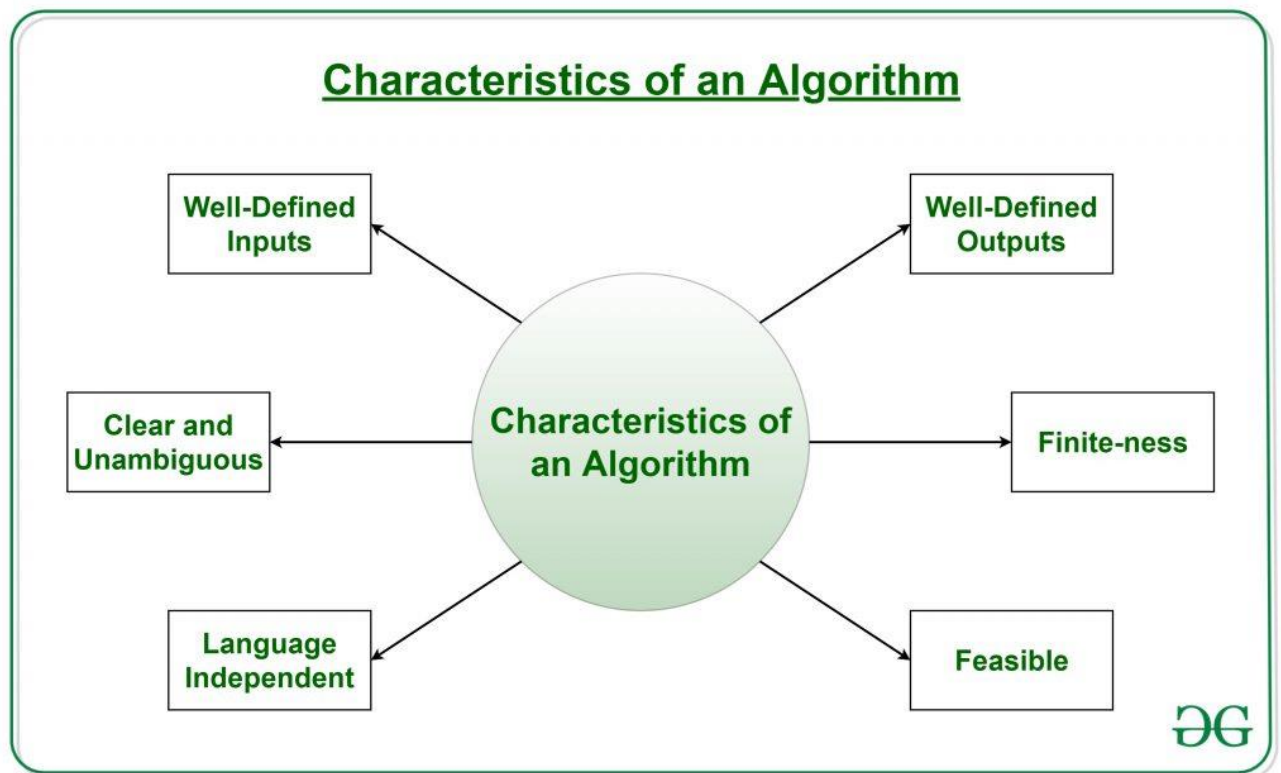
1. **Computer Science:** Algorithms form the basis of computer programming and are used to solve problems ranging from simple sorting and searching to complex tasks such as artificial intelligence and machine learning.
2. **Mathematics:** Algorithms are used to solve mathematical problems, such as finding the optimal solution to a system of linear equations or finding the shortest path in a graph.
3. **Operations Research:** Algorithms are used to optimize and make decisions in fields such as transportation, logistics, and resource allocation.
4. **Artificial Intelligence:** Algorithms are the foundation of artificial intelligence and machine learning, and are used to develop intelligent systems that can perform tasks such as image recognition, natural language processing, and decision-making.
5. **Data Science:** Algorithms are used to analyze, process, and extract insights from large amounts of data in fields such as marketing, finance, and healthcare.

These are just a few examples of the many applications of algorithms. The use of algorithms is continually expanding as new technologies and fields emerge, making it a vital component of modern society.

What is the need for algorithms?

1. Algorithms are necessary for solving complex problems efficiently and effectively.
2. They help to automate processes and make them more reliable, faster, and easier to perform.
3. Algorithms also enable computers to perform tasks that would be difficult or impossible for humans to do manually.
4. They are used in various fields such as mathematics, computer science, engineering, finance, and many others to optimize processes, analyze data, make predictions, and provide solutions to problems.

What are the Characteristics of an Algorithm?



For some instructions to be an algorithm, it must have the following characteristics:

- **Clear and Unambiguous:** The algorithm should be unambiguous. Each of its steps should be clear in all aspects and must lead to only one meaning.
- **Well-Defined Inputs:** If an algorithm says to take inputs, it should be well-defined inputs. It may or may not take input.
- **Well-Defined Outputs:** The algorithm must clearly define what output will be yielded and it should be well-defined as well. It should produce at least 1 output.
- **Finiteness:** The algorithm must be finite, i.e. it should terminate after a finite time.
- **Feasible:** The algorithm must be simple, generic, and practical, such that it can be executed with the available resources. It must not contain some future technology or anything.
- **Language Independent:** The Algorithm designed must be language-independent, i.e. it must be just plain instructions that can be implemented in any language, and yet the output will be the same, as expected.
- **Effectiveness:** An algorithm must be developed by using very basic, simple, and feasible operations so that one can trace it out by using just paper and pencil.

Factors of an Algorithm

The following are the factors that we need to consider for designing an algorithm:

- **Modularity:** If any problem is given and we can break that problem into small- small modules or small-small steps, which is a basic definition of an algorithm, it means that this feature has been perfectly designed for the algorithm.
- **Correctness:** The correctness of an algorithm is defined as when the given inputs produce the desired output, which means that the algorithm has been designed algorithm. The analysis of an algorithm has been done correctly.
- **Maintainability:** Here, maintainability means that the algorithm should be designed in a very simple structured way so that when we redefine the algorithm, no major change will be done in the algorithm.
- **Functionality:** It considers various logical steps to solve the real-world problem.
- **Robustness:** Robustness means that how an algorithm can clearly define our problem.
- **User-friendly:** If the algorithm is not user-friendly, then the designer will not be able to explain it to the programmer.
- **Simplicity:** If the algorithm is simple then it is easy to understand.
- **Extensibility:** If any other algorithm designer or programmer wants to use your algorithm then it should be extensible.

Properties of Algorithm:

- It should terminate after a finite time.
- It should produce at least one output.
- It should take zero or more input.
- It should be deterministic means giving the same output for the same input case.
- Every step in the algorithm must be effective i.e. every step should do some work.

Advantages of Algorithms:

- It is easy to understand.
- An algorithm is a step-wise representation of a solution to a given problem.
- In an Algorithm the problem is broken down into smaller pieces or steps hence, it is easier for the programmer to convert it into an actual program.

Disadvantages of Algorithms:

- Writing an algorithm takes a long time so it is time-consuming.
- Understanding complex logic through algorithms can be very difficult.
- Branching and Looping statements are difficult to show in Algorithms(**imp**).

How to Design an Algorithm (Dataflow)?

To write an algorithm, the following things are needed as a pre-requisite:

1. The **problem** that is to be solved by this algorithm i.e. clear problem definition.
2. The **constraints** of the problem must be considered while solving the problem.
3. The **input** to be taken to solve the problem.
4. The **output** is to be expected when the problem is solved.
5. The **solution** to this problem is within the given constraints.

Example: Consider the example to add three numbers and print the sum.

Step 1: Fulfilling the pre-requisites

As discussed above, to write an algorithm, its prerequisites must be fulfilled.

1. **The problem that is to be solved by this algorithm:** Add 3 numbers and print their sum.
2. **The constraints of the problem that must be considered while solving the problem:** The numbers must contain only digits and no other characters.
3. **The input to be taken to solve the problem:** The three numbers to be added.
4. **The output to be expected when the problem is solved:** The sum of the three numbers taken as the input i.e. a single integer value.
5. **The solution to this problem, in the given constraints:** The solution consists of adding the 3 numbers. It can be done with the help of the '+' operator, or bit-wise, or any other method.

Step 2: Designing the algorithm

Now let's design the algorithm with the help of the above pre-requisites:

Algorithm to add 3 numbers and print their sum:

1. START
2. Declare 3 integer variables num1, num2, and num3.
3. Take the three numbers, to be added, as inputs in variables num1, num2, and num3 respectively.
4. Declare an integer variable sum to store the resultant sum of the 3 numbers.
5. Add the 3 numbers and store the result in the variable sum.
6. Print the value of the variable sum
7. END

Step 3: Testing the algorithm by implementing it.

To test the algorithm, let's implement it in C language.

Program:

```
// C++ program to add three numbers

// with the help of above designed
// algorithm

#include <bits/stdc++.h>

using namespace std;

int main()
{
    // Variables to take the input of
    // the 3 numbers
    int num1, num2, num3;

    // Variable to store the resultant sum
    int sum;

    // Take the 3 numbers as input
    cout << "Enter the 1st number: ";
    cin >> num1;
```

```

    cout << " " << num1 << endl;

    cout << "Enter the 2nd number: ";

    cin >> num2;

    cout << " " << num2 << endl;

    cout << "Enter the 3rd number: ";

    cin >> num3;

    cout << " " << num3;

    // Calculate the sum using + operator

    // and store it in variable sum

    sum = num1 + num2 + num3;

    // Print the sum

    cout << "\nSum of the 3 numbers is: "

        << sum;

    return 0;

}

```

Output

```

Enter the 1st number: 0
Enter the 2nd number: 0
Enter the 3rd number: -1577141152

Sum of the 3 numbers is: -1577141152

```

How to analyze an Algorithm?

For a standard algorithm to be good, it must be efficient. Hence the efficiency of an algorithm must be checked and maintained. It can be in two stages:

1. Priori Analysis:

“Priori” means “before”. Hence Priori analysis means checking the algorithm before its implementation. In this, the algorithm is checked when it is written in the form of theoretical steps. This Efficiency of an algorithm is measured by assuming that all other factors, for example, processor speed, are constant and have no effect on the implementation. This is done usually by the algorithm designer. This analysis is independent of the type of hardware

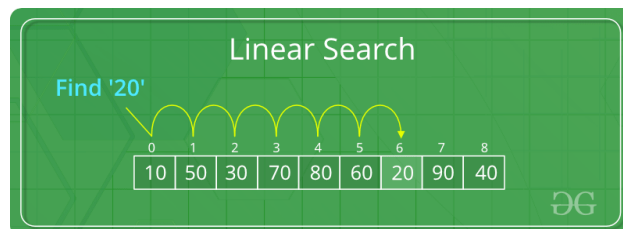
and language of the compiler. It gives the approximate answers for the complexity of the program.

2. Posteriori Analysis:

“Posterior” means “after”. Hence Posterior analysis means checking the algorithm after its implementation. In this, the algorithm is checked by implementing it in any programming language and executing it. This analysis helps to get the actual and real analysis report about correctness(for every possible input/s if it shows/returns correct output or not), space required, time consumed, etc. That is, it is dependent on the language of the compiler and the type of hardware used.

Linear Search Algorithm

Linear Search is defined as a sequential search algorithm that starts at one end and goes through each element of a list until the desired element is found, otherwise the search continues till the end of the data set.



How Does Linear Search Algorithm Work?

In Linear Search Algorithm,

- Every element is considered as a potential match for the key and checked for the same.
- If any element is found equal to the key, the search is successful and the index of that element is returned.
- If no element is found equal to the key, the search yields “No match found”.

For example: Consider the array `arr[] = {10, 50, 30, 70, 80, 20, 90, 40}` and `key = 30`

Step 1: Start from the first element (index 0) and compare **key** with each element (`arr[i]`).

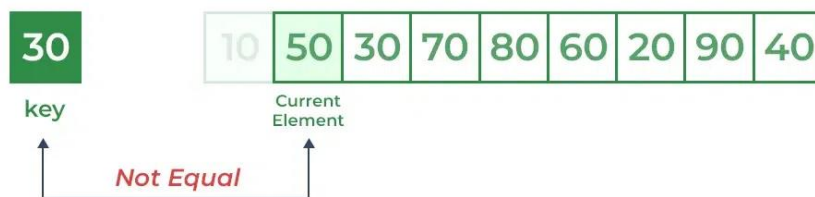
Comparing key with first element `arr[0]`. Since not equal, the iterator moves to the next element as a potential match.



Linear Search Algorithm



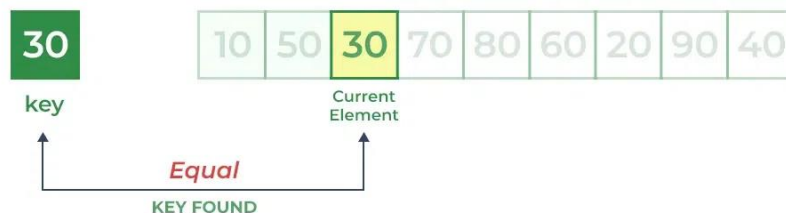
Comparing key with next element arr[1]. Since not equal, the iterator moves to the next element as a potential match.



Linear Search Algorithm



Step 2: Now when comparing arr[2] with key, the value matches. So the Linear Search Algorithm will yield a successful message and return the index of the element when key is found (here 2).



Linear Search Algorithm



Algorithm for linear search :

Let A be a linear array of n elements. The algorithm searches for the element DATA. LOC represents the location of the element DATA in the array and K is counting of steps.

The algorithm returns the value LOC=0 if the element DATA does not belong to the array.

ALGORITHM:

Step 1: Set K=1

Step 2: Set LOC=0

Step 3: Repeat steps 4 and 5 while LOC=0 and K<=n

Step 4: if A(K) = DATA then set LOC = K

Step 5: Set K=K+1

Step 6: if LOC =0 then write DATA not found in A. Else write DATA found at LOC in A

Step 7: EXIT

OR

1. **Input:** Take an array of elements (array) and a target value (target) to search for.
2. **Initialize:** Set a variable found to false and another variable index to -1.
3. **Loop through the array:**
 - Start a loop from the first element (index 0) to the last element of the array.
 - For each element at index i:
 - Check if the element at index i equals the target value.
 - If it does, set found to true and store the index i in the index variable.
 - If not, continue to the next element.
4. **Check if target is found:**
 - After the loop, check if found is true.
 - If it is true, return the index where the target was found.
 - If found is false, return -1 to indicate that the target is not present in the array.

Implementation of Linear Search Algorithm:

Below is the implementation of the linear search algorithm:

// C++ code to linearly search x in arr[].

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
int search(int arr[], int N, int x)
```

```
{
```

```
    for (int i = 0; i < N; i++)
```

```
        if (arr[i] == x)
```

```

        return i;

    return -1;
}

// Driver code
int main(void)
{
    int arr[] = { 2, 3, 4, 10, 40 };

    int x = 10;

    int N = sizeof(arr) / sizeof(arr[0]);

    // Function call

    int result = search(arr, N, x);

    (result == -1)

        ? cout << "Element is not present in array"

        : cout << "Element is present at index " << result;

    return 0;
}

```

Output

Element is present at index 3

Advantages of Linear Search:

- Linear search can be used irrespective of whether the array is sorted or not. It can be used on arrays of any data type.
- Does not require any additional memory.
- It is a well-suited algorithm for small datasets.

Drawbacks of Linear Search:

- Linear search has a time complexity of $O(N)$, which in turn makes it slow for large datasets.
- Not suitable for large arrays.

Binary Search

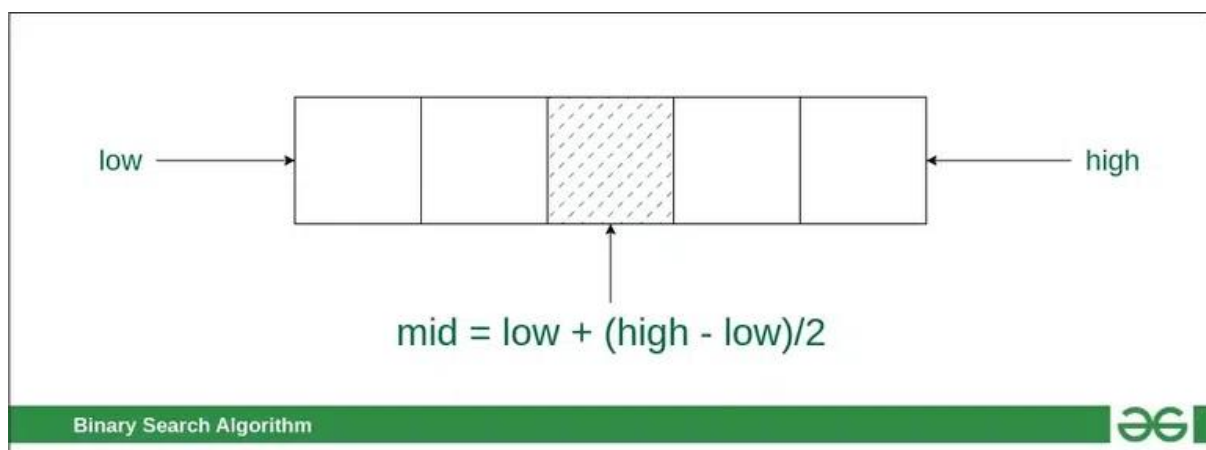
Binary Search is defined as a searching algorithm used in a sorted array by **repeatedly dividing the search interval in half**. The idea of binary search is to use the information that the array is sorted and reduce the time complexity to $O(\log N)$.



Binary Search Algorithm:

In this algorithm,

- Divide the search space into two halves by finding the middle index “mid”.



- Compare the middle element of the search space with the key.
- If the key is found at middle element, the process is terminated.

- If the key is not found at middle element, choose which half will be used as the next search space.
 - If the key is smaller than the middle element, then the left side is used for next search.
 - If the key is larger than the middle element, then the right side is used for next search.
- This process is continued until the key is found or the total search space is exhausted.

Algorithm:-

#Binary_Search(a, lower_bound, upper_bound, val) // 'a' is the given array, 'lower_bound' is the index of the first array element, 'upper_bound' is the index of the last array element, 'val' is the value to search, pos is location of key value.

Step 1: set beg = lower_bound, end = upper_bound, pos = - 1

Step 2: repeat steps 3 and 4 while beg <=end

Step 3: set mid = (beg + end)/2

Step 4: if a[mid] = val

set pos = mid

print pos

go to step 6

else if a[mid] > val

set end = mid - 1

else

set beg = mid + 1

[end of if]

[end of loop]

Step 5: if pos = -1

print "value is not present in the array"

[end of if]

Step 6: exit

How does Binary Search work?

To understand the working of binary search, consider the following illustration:

0	1	2	3	4	5	6	7	8
10	12	24	29	39	40	51	56	69

Let the element to search is, **K = 56**

We have to use the below formula to calculate the **mid** of the array -

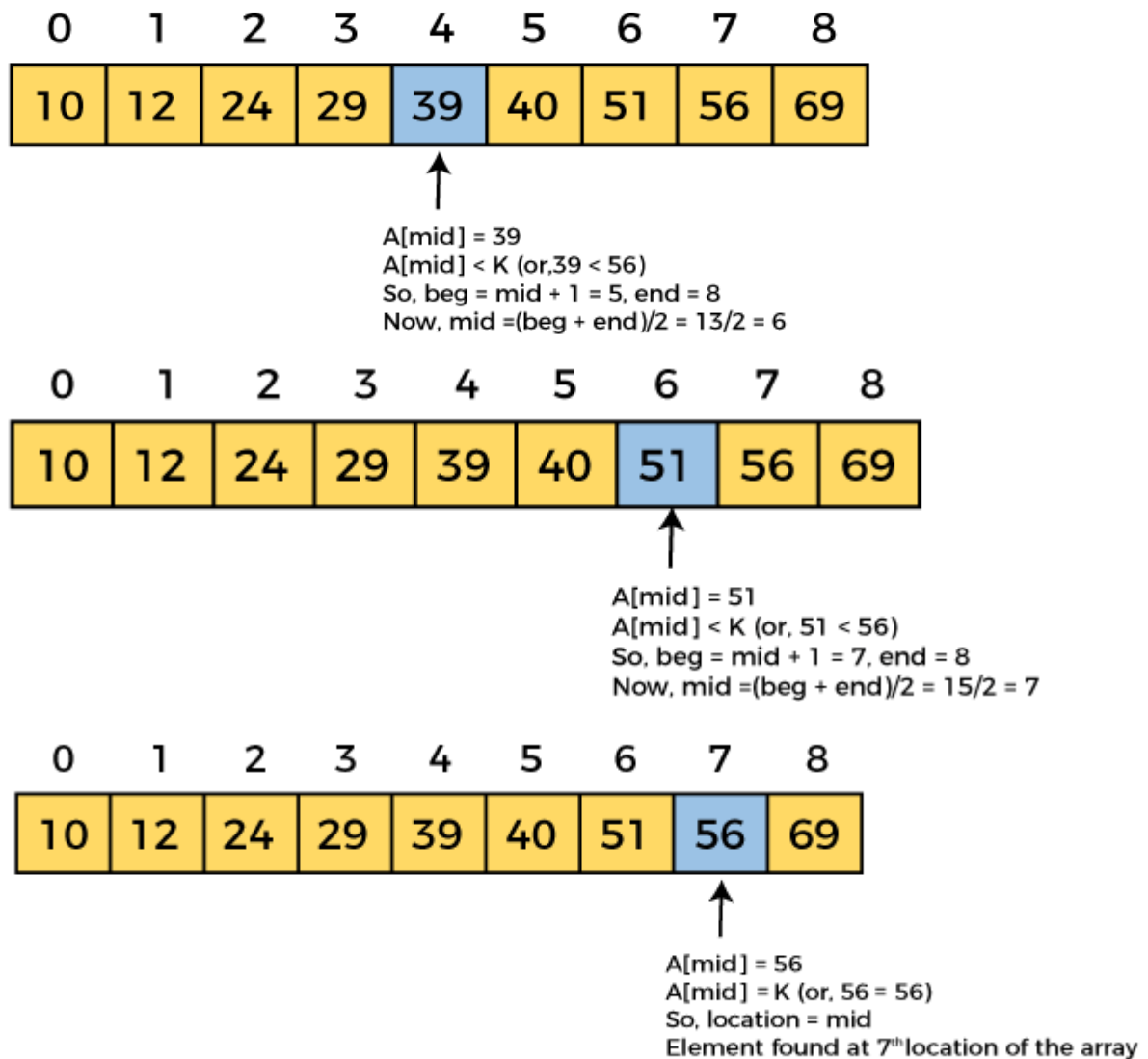
1. $\text{mid} = (\text{beg} + \text{end})/2$

So, in the given array -

beg = 0

end = 8

mid = $(0 + 8)/2 = 4$. So, 4 is the mid of the array



Now, the element to search is found. So algorithm will return the index of the element matched.

Implementation of Binary Search

Now, let's see the programs of Binary search in different programming languages.

Program: Write a program to implement Binary search in C language.

```
#include <stdio.h>

int binarySearch(int a[], int beg, int end, int val)
{
    int mid;
```

```

    if(end >= beg)

    {
        mid = (beg + end)/2;

        /* if the item to be searched is present at middle */

        if(a[mid] == val)

        {

            return mid+1;

        }

        /* if the item to be searched is smaller than middle, then it can only be in left suba
rray */

        else if(a[mid] < val)

        {

            return binarySearch(a, mid+1, end, val);

        }

        /* if the item to be searched is greater than middle, then it can only be in right sub
array */

        else

        {

            return binarySearch(a, beg, mid-1, val);

        }

    }

    return -1;

}

int main() {

    int a[] = {11, 14, 25, 30, 40, 41, 52, 57, 70}; // given array

    int val = 40; // value to be searched

    int n = sizeof(a) / sizeof(a[0]); // size of array

```



```
int res = binarySearch(a, 0, n-1, val); // Store result

printf("The elements of the array are - ");

for (int i = 0; i < n; i++)

printf("%d ", a[i]);

printf("\nElement to be searched is - %d", val);

if (res == -1)

printf("\nElement is not present in the array");

else

printf("\nElement is present at %d position of array", res);

return 0;

}
```

Output

```
The elements of the array are - 11 14 25 30 40 41 52 57 70
Element to be searched is - 40
Element is present at 5 position of array
```

Advantages of Binary Search:

- Binary search is faster than linear search, especially for large arrays.
- More efficient than other searching algorithms with a similar time complexity, such as interpolation search or exponential search.
- Binary search is well-suited for searching large datasets that are stored in external memory, such as on a hard drive or in the cloud.

Drawbacks of Binary Search:

- The array should be sorted.
- Binary search requires that the data structure being searched be stored in contiguous memory locations.
- Binary search requires that the elements of the array be comparable, meaning that they must be able to be ordered.

Difference Between Linear Search and Binary Search

Linear Search	Binary Search
In linear search input data need not to be in sorted.	In binary search input data need to be in sorted order.
It is also called sequential search.	It is also called half-interval search.
The time complexity of linear search O(n) .	The time complexity of binary search O(log n) .
Multidimensional array can be used.	Only single dimensional array is used.
Linear search performs equality comparisons	Binary search performs ordering comparisons
It is less complex.	It is more complex.
It is very slow process.	It is very fast process.